

Cómo crear cualquier app Android desde cero

Guía introductoria, independiente de cualquier ejercicio concreto

1. Para qué sirve esta guía

Esta guía explica el proceso general para arrancar cualquier proyecto Android desde absolutamente cero: instalar el programa, crear el proyecto, entender qué es cada carpeta y cada archivo, y el flujo de trabajo que se repite siempre que se añade una función nueva a una app. No está atada a ningún ejercicio en concreto — sirve como punto de partida antes de meterse con Room, con una API, o con cualquier otra funcionalidad.

2. Qué necesitas instalar

- Android Studio (incluye el SDK de Android y un JDK integrado — no hace falta instalar Java aparte).
- Una cuenta de Google (opcional, solo si vas a publicar en Play Store más adelante).
- Un ordenador con al menos 8 GB de RAM (16 GB recomendado) — el emulador consume bastante memoria.

Descarga desde developer.android.com/studio, instala con las opciones por defecto, y al abrirlo por primera vez el propio asistente descarga los componentes del SDK que faltan.

3. Crear un proyecto nuevo

1. Abrir Android Studio → "Nuevo proyecto" (o Archivo → Nuevo → Nuevo proyecto si ya tienes uno abierto).
2. Elegir una plantilla. Para empezar, la más clara es "Empty Views Activity" (sin Compose) — una sola pantalla en blanco.
3. Rellenar el formulario: Nombre, Nombre del paquete (identificador único de la app, tipo `com.empresa.nombreapp`), Lenguaje (Kotlin), SDK mínimo.
4. Pulsar Finalizar y esperar la primera sincronización de Gradle (puede tardar varios minutos la primera vez).

¿Qué es el "SDK mínimo"?

Es la versión más antigua de Android que tu app aceptará instalar. Cuanto más bajo, más dispositivos antiguos la podrán usar, pero menos funciones modernas del sistema podrás usar directamente. API 24 (Android 7.0) es un buen equilibrio para empezar.

¿Qué es el "nombre del paquete"?

Es el identificador único de la app en todo el mundo (como un DNI). Una vez publicada en Play Store, no se puede cambiar. Se suele escribir al revés del dominio de la empresa: `com.kreativascomunicacion.miapp`.

4. Anatomía de un proyecto: qué es cada cosa

Por defecto, el panel izquierdo muestra la vista "Android", que agrupa las carpetas de forma simplificada. Para ver la estructura real, cambia el desplegable de arriba de "Android" a "Proyecto".

```
MiApp/  
├── app/  
│   ├── build.gradle.kts  
│   └── src/main/
```

→ dependencias y configuración de ESTE módulo

- AndroidManifest.xml - java/... - res/ - build.gradle.kts - settings.gradle.kts - gradle.properties	→ "tarjeta de identidad" de la app → todo el código Kotlin/Java → todos los recursos (layouts, textos, colores, imágenes) → configuración a nivel de todo el proyecto → qué módulos forman el proyecto → opciones generales de compilación
Carpeta / archivo	Para qué sirve
AndroidManifest.xml	Declara qué Activities tiene la app, permisos que pide, el icono, el nombre visible...
res/layout/	Los diseños de pantalla, en XML (qué botones, textos, imágenes hay y dónde).
res/values/strings.xml	Todos los textos de la app en un solo sitio (facilita traducir la app).
res/values/themes.xml	Colores y estilos globales (tema) de la app.
res/mipmap-*/	Los iconos de la app en distintos tamaños.
res/drawable/	Imágenes, iconos vectoriales y formas reutilizables.
java/.../MainActivity.kt	El código Kotlin que da vida a una pantalla.
build.gradle.kts (módulo app)	Qué librerías externas usa la app (dependencies) y su configuración de compilación.

5. Los cuatro conceptos que hay que entender antes de programar nada

5.1 • Activity

Una Activity es, a grandes rasgos, una pantalla. Toda app empieza con al menos una (MainActivity). Cada pantalla nueva de la app suele ser una Activity nueva (o un Fragment, un concepto más avanzado para pantallas dentro de una misma Activity).

5.2 • Layout XML

El diseño visual de cada pantalla se escribe en un archivo XML dentro de res/layout/, no en Kotlin. Ahí se colocan los botones, textos, imágenes, listas... y su posición. El código Kotlin de la Activity luego "conecta" con esas vistas para leer o cambiar lo que muestran.

5.3 • Gradle

Gradle es el sistema que compila la app: descarga las librerías que se declaran en build.gradle.kts, compila el código Kotlin, empaqueta los recursos y genera el archivo final (.apk o .aab) instalable en un dispositivo. Cada vez que se añade una librería nueva (por ejemplo, Room o Glide), se declara ahí y se pulsa "Sincronizar ahora".

5.4 • Recursos (res)

Android obliga a separar el contenido (textos, colores, imágenes, tamaños) del código. Esto tiene una ventaja práctica enorme: cambiar el texto de un botón, o el color de toda la app, se hace en un solo sitio (strings.xml o themes.xml) sin tocar ni una línea de Kotlin.

6. Ejecutar la app por primera vez

1. Crear un dispositivo virtual (emulador): Herramientas → Device Manager → Crear dispositivo → elegir un modelo (por ejemplo, Pixel 6) → elegir una imagen de sistema (por ejemplo, Android 14) → Finalizar.

2. O bien, conectar un móvil físico por USB con la "Depuración USB" activada (Ajustes → Opciones de desarrollador).
3. Pulsar el botón  verde ("Ejecutar") en la barra superior.
4. Esperar a que compile e instale — la primera vez tarda más.

Cómo activar las Opciones de desarrollador en un móvil real

Ajustes → Acerca del teléfono → pulsar 7 veces seguidas sobre "Número de compilación". Aparece un mensaje confirmando que ya eres desarrollador; luego se activa "Depuración USB" dentro de Opciones de desarrollador.

7. El flujo de trabajo que se repite SIEMPRE al añadir algo nuevo

Da igual si la función nueva es "un botón que cambia de color" o "guardar datos con Room": el proceso mental es siempre el mismo, y conviene automatizarlo desde el principio:

1. Diseñar la pantalla (o el cambio) en el XML de res/layout/: qué vistas hacen falta y con qué id.
2. Abrir la Activity/Fragment en Kotlin y "conectar" con esas vistas (con ViewBinding, la forma recomendada hoy en día).
3. Escribir la lógica: qué pasa cuando el usuario toca ese botón, escribe ese texto, etc.
4. Si hace falta guardar datos, decidir dónde: memoria (se pierde al cerrar), Room (base de datos local), o una API remota (Retrofit).
5. Ejecutar y probar en el emulador o el móvil.
6. Si algo falla, mirar el Logcat (pestaña inferior de Android Studio): ahí aparece el motivo exacto del error, casi siempre con la línea de código responsable.

El Logcat es tu mejor amigo

Cuando una app "se cierra sola" (crashea), el motivo exacto siempre está en el Logcat, normalmente en rojo y con la palabra "FATAL EXCEPTION". Acostumbra al alumnado a leerlo de arriba abajo desde el principio: ahí está la respuesta al 90% de los errores.

8. Buenas prácticas desde el primer día

- Usar Git desde el principio (VCS → Enable Version Control), aunque sea un proyecto pequeño de clase.
- Nombrar los ids de las vistas de forma consistente: ivFoto (ImageView), tvTitulo (TextView), btnGuardar (Button), etAlgo (EditText).
- Separar el diseño visual (XML) de la lógica (Kotlin): evitar meter textos o colores "a fuego" en el código.
- Activar ViewBinding desde el build.gradle.kts del módulo app en cuanto se crea el proyecto (buildFeatures { viewBinding = true }); ahorra muchísimos errores de findViewById.
- Probar en un emulador con una versión de Android distinta a la del móvil personal, para detectar pronto problemas de compatibilidad.

9. Glosario rápido

Término	Qué es
APK / AAB	El archivo final instalable de la app (AAB es el formato que exige hoy Play Store).
Gradle Sync	El proceso de descargar dependencias y preparar el proyecto para compilar.
Emulador / AVD	Un dispositivo Android virtual que corre en el ordenador.

Manifest	El archivo AndroidManifest.xml, con la configuración global de la app.
Layout	El diseño visual de una pantalla, en XML.
ViewBinding	Mecanismo para acceder a las vistas de un layout desde Kotlin sin findViewById.
Logcat	El panel de registro donde aparecen los mensajes y errores de la app en ejecución.
minSdk / targetSdk / compileSdk	Versión mínima soportada / versión objetivo probada / versión del SDK usada para compilar.

10. Y a partir de aquí, ¿qué?

Con esto ya se puede crear cualquier app sencilla de una sola pantalla. Los siguientes pasos naturales, cada uno con su propia guía específica según lo que se necesite:

- Guardar datos de forma permanente → Room (base de datos local).
- Consumir una API externa → Retrofit.
- Varias pantallas dentro de una sola Activity → Fragments + Navigation Component.
- Listas de elementos → RecyclerView.
- Interfaces modernas declarativas → Jetpack Compose (alternativa a los layouts XML).